

Borg El Arab Technological University

Faculty of Industrial Energy Technology

Department of Information Technology - Software Technology

Client-Server Real-Time Chat Application

TCPChat603JFX

Course	Network Programming
Semester	2nd Term, Academic Year 2025 / 2026
Lecturer	Dr. Aya Ibrahim
Student Name	Maryam Eid Abd Elsalam
Student ID	2320603
Assigned Port	20603

Abstract

This report documents the design and implementation of TCPChat603JFX, a fully self-contained real-time desktop chat application developed as the culminating project of a network programming course at Borg El Arab Technological University. The system is written entirely in Java and JavaFX and requires no external runtime dependencies beyond the JDK and JavaFX SDK. On startup it automatically binds a multi-threaded TCP server to port 20603, then opens a split-pane graphical window through which one or two human users can chat in real time. An integrated bot manager can inject up to four automated participants that respond naturally after brief silence windows. The application covers a broad range of network programming techniques: socket lifecycle management, a custom line-delimited text protocol, concurrent client handling, automatic reconnection with visible status feedback, and server throughput measurement. Its 19th-century naturalist illustration aesthetic — parchment tones, serif typography, and engraved-style borders — gives the interface a cohesive and distinctive visual character.

System Architecture

TCPChat603JFX follows a client-server model where server and clients coexist inside a single Java process. The Main class launches a daemon thread that binds a ChatServer instance to port 20603, then waits three hundred milliseconds before the JavaFX application thread opens the graphical window. This self-hosting approach means the user never needs to run a separate process; the entire system starts and stops as one unit.

The server uses a `CachedThreadPool`, dedicating one thread to each connected client. Each thread runs a `ClientHandler` that performs a JOIN handshake on connection, relays subsequent messages to every registered client via a synchronised broadcast method, and cleans up the client map on disconnect. A separate single-thread `ScheduledExecutorService`, also running as a daemon, logs message throughput to the console every five seconds when traffic is active.

Client logic lives in the `ChatClient` class, which runs a reconnecting connection loop on a background daemon thread. Two `ChatClient` instances can coexist in the same window — one per human chat pane. Bot participants are additional `ChatClient` instances created and managed by `BotManager`. All instances communicate over loopback with the embedded server, making the system entirely self-contained on a single machine.

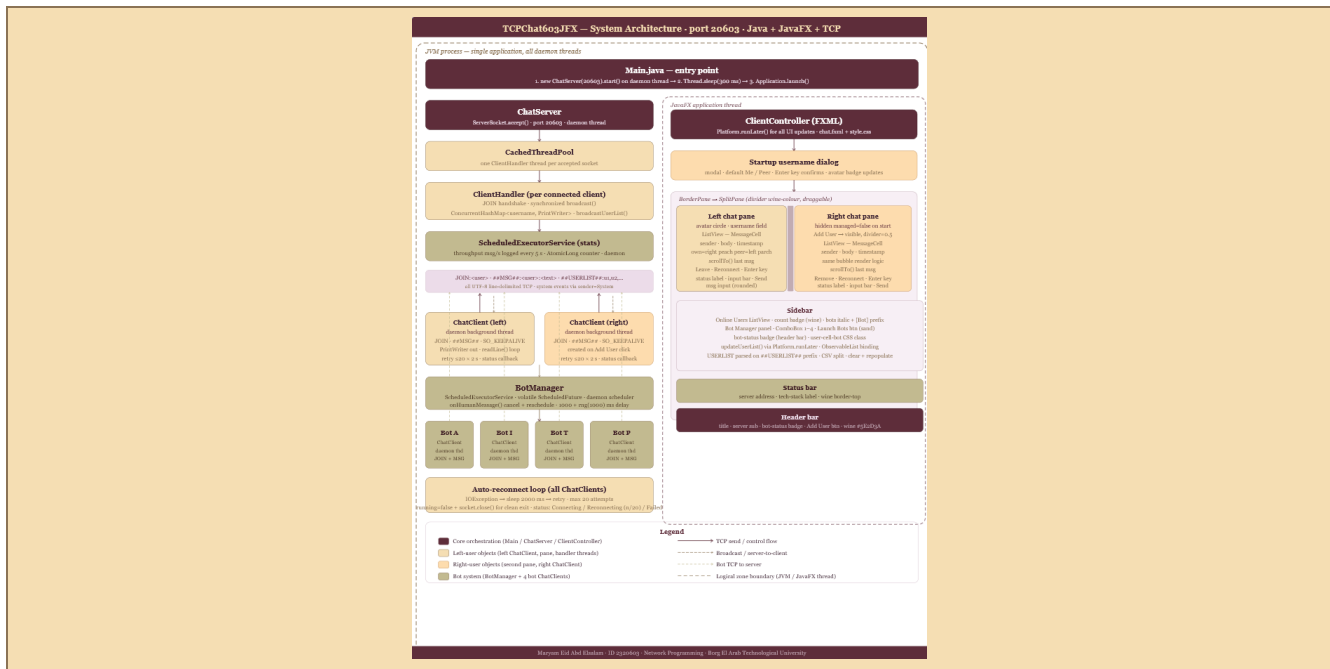


Figure 1. Class structure and thread topology of TCPChat603JFX.

Class Responsibilities

Class	Role
Main	Entry point. Starts the server daemon thread, then launches JavaFX.
ChatServer	TCP server on port 20603. CachedThreadPool for concurrent clients. Handles broadcast, user-list updates, and throughput stats.
ChatClient	Socket model for human panes and bots. JOIN handshake, message sending, auto-reconnect loop (up to 20 retries × 2 s).
ClientController	JavaFX FXML controller. Manages both chat panes, sidebar, online-users list, bot panel, and the startup dialog.
BotManager	Creates bot ChatClient instances and schedules a single pending reply 1–2 s after each human message, cancelling any prior pending reply.

Communication Protocol

All messages are transmitted as plain UTF-8 text lines over TCP. Three protocol prefixes govern every exchange between clients and server.

Direction	Format	Purpose
Client → Server	JOIN:<username>	First line sent; registers the user on the server.
Client → Server	##MSG##:<username>:<text>	Sends a chat message to be broadcast.
Server → Client	##MSG##:<username>:<text>	Delivers a chat message to all connected clients.
Server → Client	##USERLIST##:<u1>,<u2>,...	Pushes the current participant list after any membership change.

System events — join and leave notifications — use the reserved sender name **System** and travel through the same **##MSG##** channel, so no additional parsing branch is needed on the client. If a JOIN line carries an empty username, the server substitutes a **Guest_** fallback with a random three-digit suffix to ensure every participant has a non-empty display name.

Implemented Features

1. Embedded Multithreaded TCP Server

ChatServer binds a **ServerSocket** on port 20603 and blocks on **accept()** in an infinite loop. Every accepted socket is handed to a **ClientHandler** submitted to a **CachedThreadPool**, so the server handles as many concurrent connections as the JVM and OS allow. The shared client map is a **ConcurrentHashMap**, and both **broadcast** and **broadcastUserList** are synchronised to prevent partial updates during concurrent membership changes. A separate daemon **ScheduledExecutorService** samples an **AtomicLong** message counter every five seconds and prints a messages-per-second figure alongside the current client count; the counter is atomically reset after each sample.

2. Socket Client with Auto-Reconnect

ChatClient encapsulates all socket logic in a **connectionLoop** that runs on a daemon background thread. On first entry it emits a Connecting status, opens a TCP socket, sets the **SO_KEEPALIVE** option, writes the JOIN handshake, and enters a line-by-line read loop. If the socket throws an **IOException** — or if the server closes the connection — the loop catches the exception, increments a retry counter, emits a **Reconnecting (n/20)** status, sleeps for two seconds, and retries. After twenty failed attempts it emits Connection failed and stops. The **disconnect** method sets a volatile **running** flag to false and closes the socket, causing any blocked **readLine** to unblock and the loop to exit cleanly.



Figure 2. Reconnecting status label — visible during server unavailability.

3. Graphical Interface Layout

The main window is a JavaFX BorderPane loaded from an FXML file. The top region holds a header bar styled in wine (#5E2D3A) that displays the application name, the server address, a bot-status badge, and an Add User button. The center region contains a fixed-width sidebar (190–210 px) and a horizontally growing SplitPane. The bottom region holds a status bar confirming the server address and technology stack.

The sidebar is divided into two sections. The upper section shows a live Online Users list with a count badge. The lower Bot Manager panel contains a ComboBox for selecting the number of bots (1–4), a Launch Bots button, and a status label. The SplitPane starts with its divider at position 1.0, meaning only the left chat pane is visible. When the user clicks Add User, the right pane becomes managed and visible and the divider moves to 0.5.

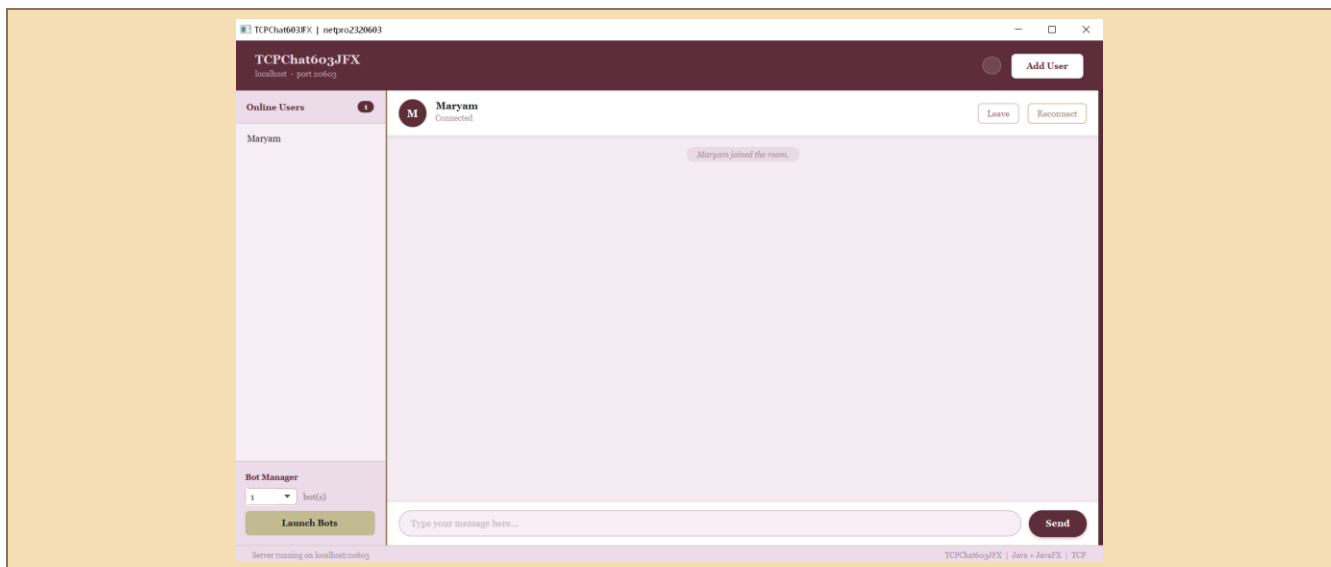


Figure 3. Main window — single-user mode with sidebar and online-users list.

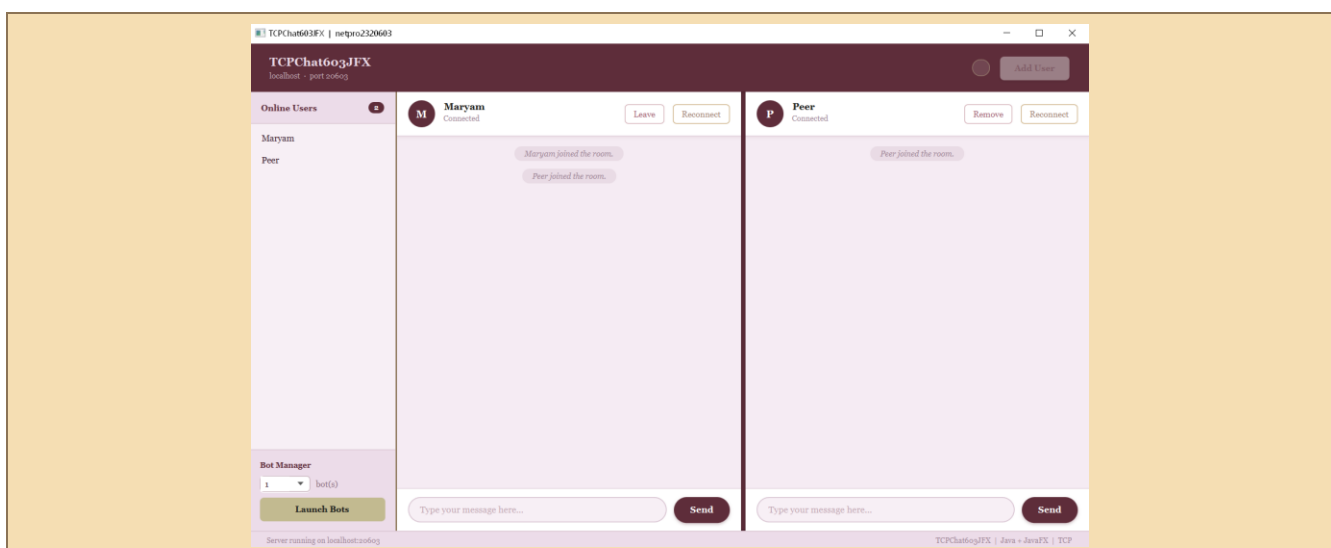


Figure 4. Split-pane mode with two simultaneous human participants.

4. Message Bubble Rendering

Messages are rendered by a custom **MessageCell** (a **ListCell** subclass). System messages — join and leave notifications — are wrapped in a centred HBox with a lightly tinted rounded background and displayed in italics, visually separating administrative events from conversation content.

Chat messages are shown as styled VBox bubbles. Each bubble contains three elements: a small bold serif sender-name label, the message body with word-wrap enabled, and a muted timestamp label right-aligned at the bottom. Own messages use a warm peach background (#FDDCAE) with a terracotta border and align to the right; peer messages use a parchment fill (#F5DEB3) with a brown border and align to the left. After each cell update, the ListView scrolls to the last item so new messages are always visible.

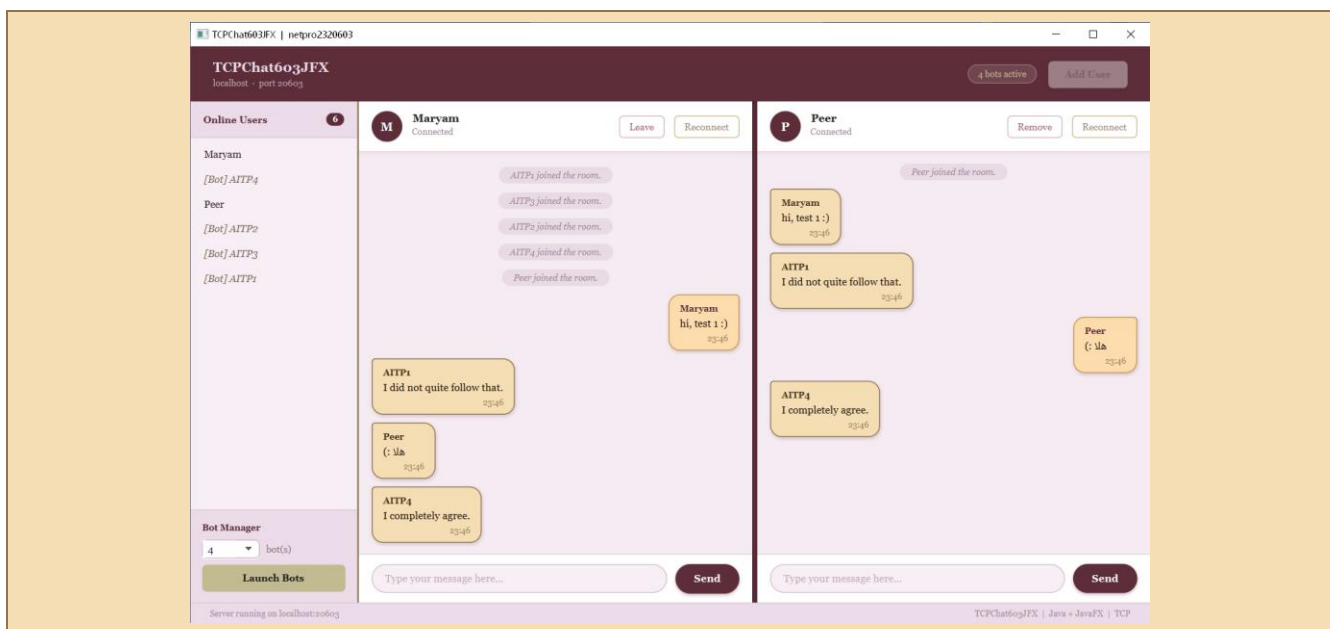


Figure 5. Message bubbles — own messages (right, peach) and peer messages (left, parchment).

5. Pre-Chat Username Dialog

Before the main window is populated, Platform.runLater schedules a modal Dialog that prompts both participants to enter their names. Two TextField controls — pre-filled with Me and Peer respectively — let users either proceed immediately with the defaults or type custom names. On confirmation, both username fields update, the avatar badges refresh to show the first letter of each name, and the left client connects. Pressing Enter in either dialog field has the same effect as clicking OK.

6. Automated Bot System

BotManager creates between one and four **ChatClient** instances that connect to the server exactly as human clients do, using the fixed identifiers A, I, T, and P. Each bot runs its own background daemon thread through **ChatClient**'s standard connection loop, so bots appear in the online-users list, receive all broadcast messages, and can send messages through the same **##MSG##** channel. In the

online-users list, bot names are rendered in italic with a [Bot] prefix, making it easy to distinguish automated from human entries.

The scheduling logic uses a single-thread `ScheduledExecutorService` whose thread is also a daemon. A volatile `ScheduledFuture` field holds the pending reply task. When a human sends a message, `onHumanMessage` cancels any existing pending future and schedules a new `sendBotMessage` call to execute between one and two seconds later. Because the future is always cancelled before a new one is scheduled, at most one bot reply is ever pending at a time, preventing message flooding during rapid conversation.

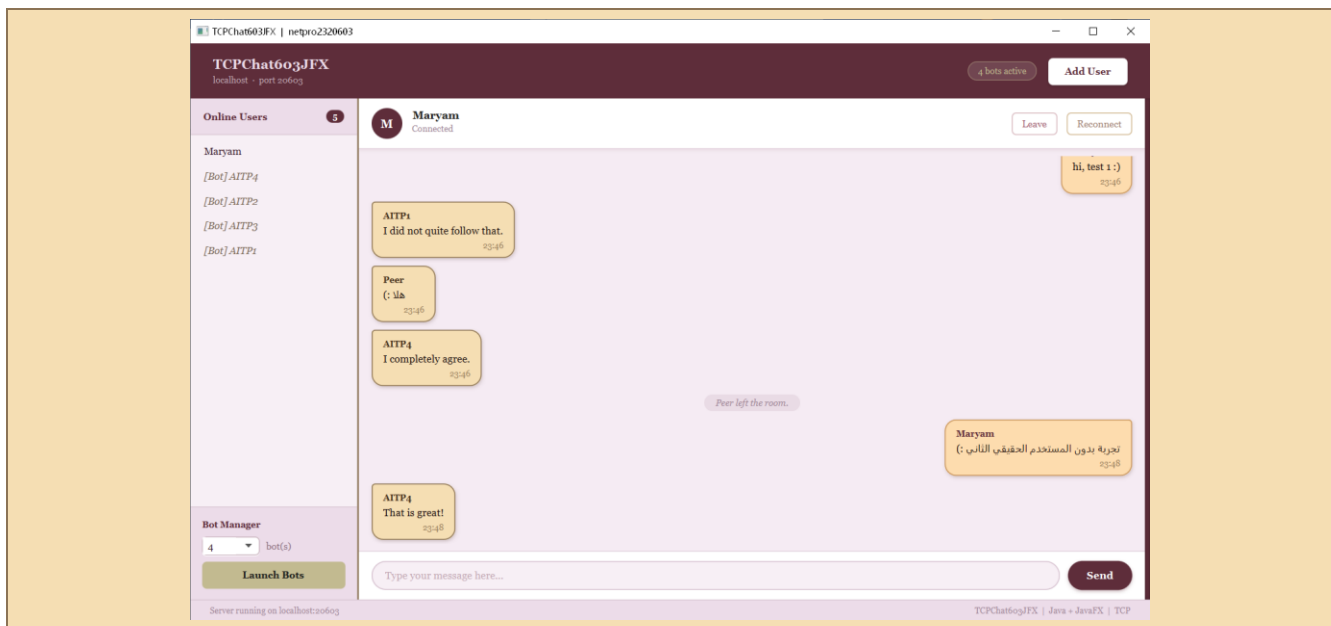


Figure 6. Bot participants active — italic [Bot] labels in the online-users list.

7. Keyboard Shortcut for Sending

Both input text fields are wired to an `onKeyPressed` handler that checks for the ENTER key code. Pressing Enter triggers the same action as clicking the corresponding Send button, allowing the user to type and send messages without reaching for the mouse. The input field is cleared after every successful send, and if a `BotManager` is active, `onHumanMessage` is called to reset the bot reply schedule.

8. Leave and Reconnect Controls

Each pane header carries two buttons. The Leave button calls `disconnect` on the client, clears the message list, and updates the status label to Disconnected. The Reconnect button creates a fresh `ChatClient` with the current username and starts its connection loop, allowing a user to rejoin the server after voluntarily leaving or after a connection failure without restarting the application.

9. Live Online-Users Sidebar

Every time a client joins or leaves, `ChatServer` broadcasts an updated `##USERLIST##` line containing all current usernames as a comma-separated string. On the client side, `updateUserList` clears the `ObservableList` bound to the online-users ListView, splits the CSV payload, and repopulates it. The count badge in the sidebar header is updated to reflect the new total. Bot entries appear in muted italic, while human entries use standard dark text.

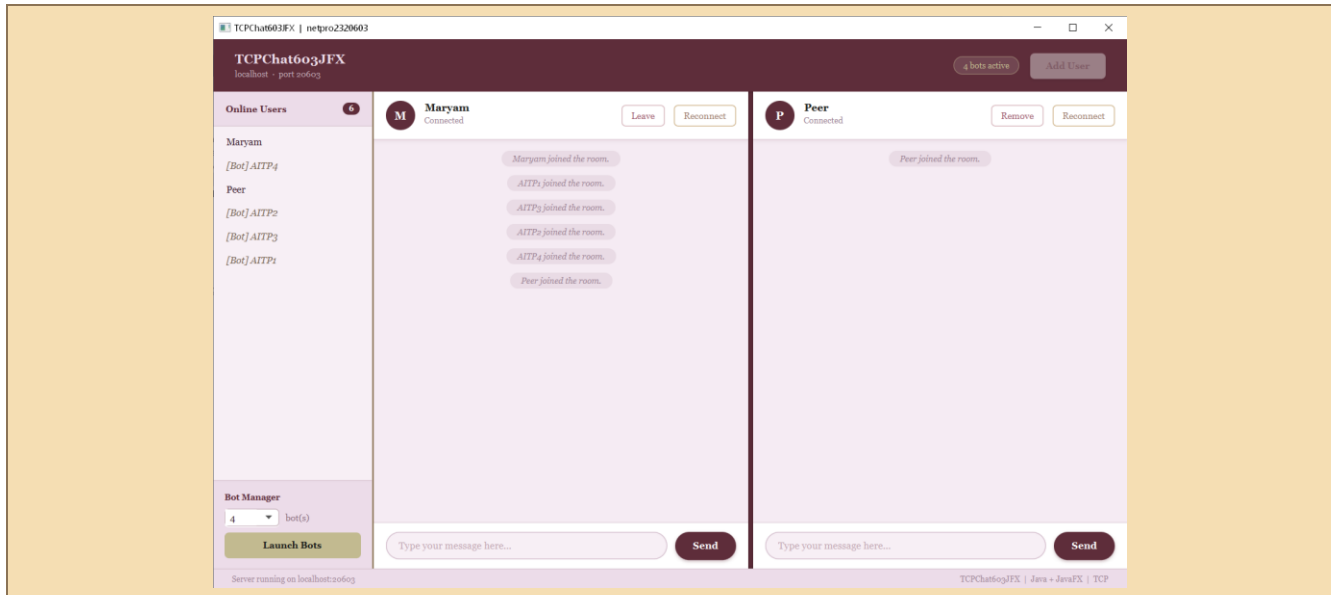


Figure 7. Live online-users sidebar with bot entries displayed in italic.

Technologies Used

- **Java SE** — Core language, standard library, java.net socket API
- **JavaFX** — Graphical interface, FXML layout, ObservableList data binding, CSS theming
- **TCP Sockets (java.net.Socket / ServerSocket)** — Transport layer with SO_KEEPALIVE
- **Java Multithreading** — CachedThreadPool (server), daemon threads (client, bots, scheduler)
- **ConcurrentHashMap / AtomicLong** — Thread-safe shared state without explicit locking on reads
- **ScheduledExecutorService** — Throughput logging (server) and bot reply scheduling (BotManager)
- **CSS (JavaFX dialect)** — Full visual theming, bubble shapes, avatar circles, scrollbar styling

Visual Design and User Experience

The interface draws deliberate inspiration from 19th-century naturalist and scientific atlas illustration. The colour palette centres on warm parchment tones — `#F5DEB3` for received-message bubbles and `#F7EFF5` for the sidebar — offset by a deep burgundy wine (`#5E2D3A`) that appears in the header bar, avatar circles, the Send button, and all section borders. A muted sand-gold (`#C2BA90`) accents the bot launch button and the bot-status badge. Typography uses Georgia and Book Antiqua throughout — the same serif faces found on Victorian botanical plate captions — and is applied globally via the `.root` CSS rule so no element falls back to a system sans-serif.

Colour Reference

The painting below served as the primary colour reference during the design phase. Its warm burgundy petals, ivory-cream blooms, antique bronze vase, and dusty sage foliage map directly onto the six interface tones listed in Table 1. Selecting a real-world natural source ensured the palette remained coherent and visually harmonious rather than arbitrary.



Figure 8. Victorian botanical still life — primary colour reference for the TCPChat603JFX palette.

Table 1 — Interface Colour Tokens

Swatch	Hex Token	Role in the Interface
	<code>#5E2D3A</code>	Burgundy Wine — header bar, avatars, Send button, section borders
	<code>#ECDCE9</code>	Blush Lavender — primary application background
	<code>#F5DEB3</code>	Warm Parchment — received-message bubbles
	<code>#F7EFF5</code>	Ivory Rose — sidebar panel background
	<code>#C2BA90</code>	Sand Gold — bot launch button and status badge accent
	<code>#8B7355</code>	Antique Brown — borders, dividers, and secondary outlines

Table 1. The six colour tokens and their roles in the stylesheet.

The message bubble shapes reinforce spatial meaning through corner radii alone. The 14 4 14 14 px radius on own messages creates a flat top-right corner that reads as a speech-bubble tail pointing toward the right side of the screen; the 4 14 14 14 px radius on peer messages mirrors this with a flat top-left corner. This convention lets the user understand the direction of each message without reading the sender name.

Bot users appear in italic in the online-users list, a typographic convention borrowed from editorial marginalia that signals a secondary or automated voice without requiring an icon. The status labels, timestamps, and status-bar text all use the same muted mauve (#A08898) that atlas illustrators used for annotation text — visible but subordinate to the main content.

Challenges and Solutions

Concurrent Modification of the Client Map

The server handles each client on its own thread, so two handlers could simultaneously register a new entry and broadcast the updated user list, producing an inconsistent snapshot visible to some clients. Using `ConcurrentHashMap` prevents structural corruption on concurrent reads and writes, while synchronising `broadcast` and `broadcastUserList` on the server instance ensures that any in-progress delivery completes before the map is modified or re-read.

JavaFX Thread Safety

JavaFX requires all scene-graph modifications to execute on its application thread, yet all socket reads occur on background threads. Every callback that touches a UI element — appending a `ChatMessage` to an `ObservableList`, updating a status `Label`, refreshing the online-users list — is therefore wrapped in `Platform.runLater`. This queues the update to execute at the next JavaFX pulse without blocking the calling background thread, keeping the interface responsive throughout.

Well-Behaved Bot Reply Scheduling

`BotManager` resolves the naive-bot problem with a single volatile `ScheduledFuture`. Every new human message cancels the current future before scheduling a fresh one, so at most one reply is ever pending and it is always timed relative to the most recent human activity. The one-to-two-second randomised delay prevents the bot from feeling mechanical while still ensuring a response arrives within a predictable window.

Clean Thread Shutdown

All threads created by `ChatClient`, `ChatServer`, and `BotManager` are marked as daemon threads. Daemon threads are terminated automatically when all non-daemon threads finish, so closing the JavaFX window is sufficient to stop the entire application without an explicit shutdown hook.

Network Configuration Reference

Parameter	Value
Protocol	TCP (Transmission Control Protocol)
Server Port	20603
Character Encoding	UTF-8 (explicit on all streams)
Socket Option	SO_KEEPALIVE enabled on each client socket
Max Reconnect Attempts	20
Reconnect Interval	2,000 ms between attempts
Bot Reply Delay	1,000 – 2,000 ms (randomised per message)
Bot Count Range	1 – 4 concurrent bots
Throughput Log Interval	Every 5 seconds (server console)
Thread Model	Daemon threads throughout (server pool, client, bots, scheduler)

Conclusion

Developing TCPChat603JFX provided sustained, practical experience across several interconnected areas of software engineering. Designing a custom text protocol clarified the importance of unambiguous message framing and of separating control traffic from user content at the protocol level rather than the application level. Managing multiple concurrent connections reinforced the value of thread-safe data structures and the care required when sharing mutable state.

Integrating asynchronous network callbacks with JavaFX's single-threaded rendering model demonstrated in practice why a clean boundary between the I/O layer and the presentation layer matters: any blurring of that boundary produced freezes whose root cause was distant from the symptom. The reconnection loop introduced the broader lesson that production-grade network code must treat connection failure as an ordinary event — not an exception — and provide the user with visible, actionable feedback throughout.

The bot scheduling problem was an exercise in thinking about concurrency at the design level before writing a line of code: the decision to hold a single cancellable future rather than a queue of futures resolved the flooding problem structurally rather than through defensive logic. Finally, applying the 19th-century atlas theme showed that a coherent visual language simplifies design decisions: once the colour palette, typeface, and border conventions were fixed, every subsequent UI choice had a clear precedent to follow.